```
$ ./whyrunner --tcc-ii --parcial
```

WhyRunner

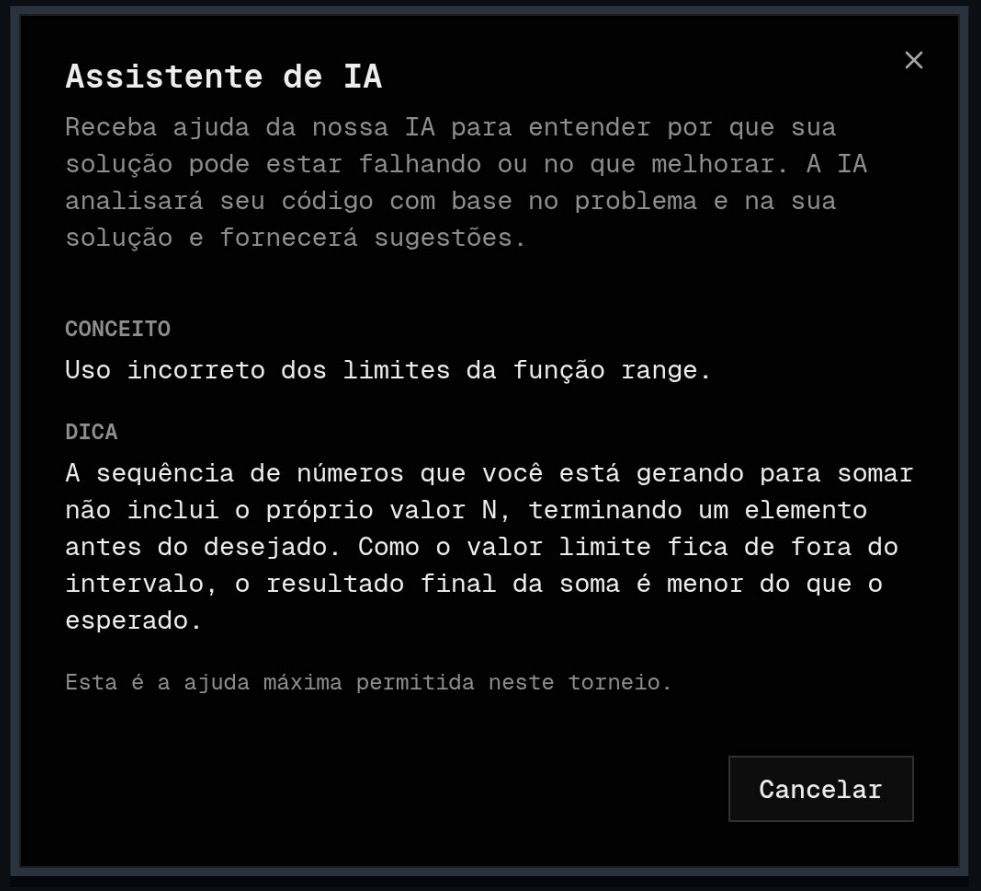
Juiz online para o ensino de programação, com dicas de IA que o professor controla

Juan Israel dos Anjos

Orientadora: Suellen Mapa

Bacharelado em Sistemas de Informação · IFMG Ouro Branco

`TCC II · Apresentação parcial · [data]`



Dica da IA em dois níveis, dentro de um torneio

Juízes online corrigem código. Ensinar é outra coisa

Juiz online tradicional

✗ Wrong Answer

Caso de teste 3 de 10 · sem explicação

BOCA, Run Codes: veredito binário, pensado para competição

Motivação: anos de maratona de programação usando esses sistemas como aluno.

Escopo do TCC

FAZ

- Torneios
- Turmas e aulas
- Execução isolada · 6 linguagens
- Dicas de IA graduadas
- Restrições de solução

NÃO FAZ

- Detecção de plágio
- Avaliar estilo de código
- Substituir a avaliação do professor

O aluno recebe "Wrong Answer" e para por aí

Problema

Juízes priorizam justiça e velocidade. O feedback é um veredito binário.

Justificativa

Erro sem explicação não vira aprendizado. E o professor não consegue montar atividades nem exigir uma abordagem.

PERGUNTA DE PESQUISA

Feedback por IA e atividades curadas pelo professor tornam um juiz online mais útil para o ensino?

Objetivo geral: desenvolver e avaliar um juiz online voltado ao ensino de programação.

1. Executar código com segurança em várias linguagens
2. Dar dicas por IA sem entregar a solução
3. Deixar o professor montar aulas e exigir algoritmos
4. Avaliar a usabilidade com alunos e professor

● em andamento

Nenhum dos sistemas analisados ensina, só corrige

Sistema	Feedback IA	Linguagens	Tempo real	Aulas curadas	Restrições
Ataux (2021)	—	Limitadas	—	—	—
GOJ – UnB (2022)	—	Moderadas	Limitado	—	—
Run Codes	—	Muitas	●	—	—
BOCA (2004)	—	Moderadas	●	—	—
WhyRunner	●	6	●	●	●

Diferencial: dicas por IA, aulas montadas pelo professor e restrições de solução: verificar *como* o aluno resolveu, não só se a saída está certa.

Ciclos iterativos: cada versão corrigiu a anterior

	TENTADO		ADOTADO	POR QUÊ
Fila	~~BullMQ → AWS SQS~~	→	**PostgreSQL LISTEN/NOTIFY**	sem polling ocioso nem outro serviço pago
Backend	~~API NestJS separada~~	→	**Next.js Server Actions**	tipos e autenticação não ficam duplicados
Banco	~~Supabase~~	→	**PostgreSQL + Drizzle**	liberdade para lógica no servidor
Sandbox	~~Docker por submissão~~	→	**isolate (usado na IOI)**	Docker exigia host privilegiado

Levantamento: literatura + conversas com alunos + experiência própria

MVP testado informalmente no Discord

Dois serviços que só conversam pelo banco



1. Aluno submete em problema, torneio ou aula
2. Server Action grava PENDING e dispara NOTIFY
3. Judge recebe e busca os casos de teste
4. isolate compila e executa com limites
5. Checa restrições: análise estática, depois IA
6. Veredito gravado; aluno notificado

```
Command::new("isolate")
    .arg(format!("--time={}", limits.time_secs))
    .arg(format!("--mem={}", limits.mem_kb))
    .arg(format!("--processes={}", limits.processes))
```

`judge/src/isolate.rs`

Cada execução roda numa caixa sem rede, com limite de tempo, memória e processos.

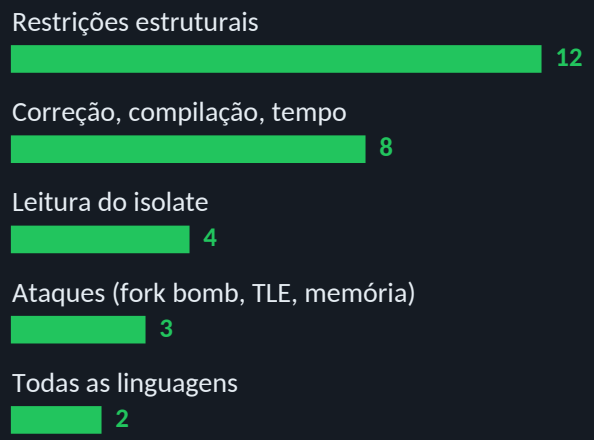
C · C++ · Java · Python · Rust · Portugol

Como o trabalho é validado

FEITO

Validação técnica

29 testes automatizados no judge



FEITO

Validação dos problemas

Antes de publicar, a solução de referência do professor roda no judge contra todos os casos de teste.

Mudou um teste? Publicação bloqueada até rodar de novo.

PLANEJADO

Avaliação com usuários

Aplicação
torneio + aula com uma turma real

Usabilidade
questionário SUS (nota 0–100)

Uso
análise descritiva de veredictos, tempo de correção e uso das dicas

Três lugares para submeter, três propósitos

PROBLEMA



Para quê
Treino livre no banco de problemas

Quem monta
Autor publica; aluno resolve no seu ritmo

IA
Sem IA: veredito e casos de exemplo

TORNEIO

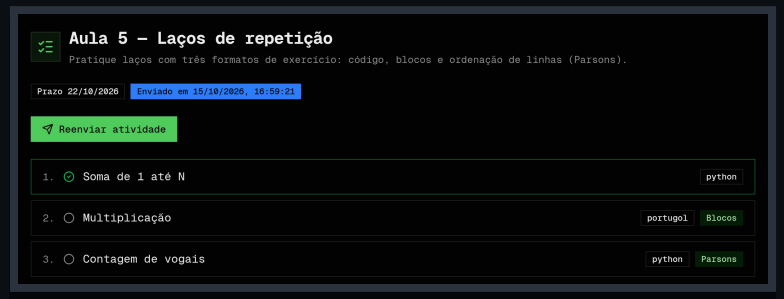


Para quê
Competir ou avaliar com tempo marcado

Quem monta
Professor define problemas, período e acesso

IA
IA no nível que o professor escolher

AULA



Para quê
Aprender em sequência, com prazo e nota

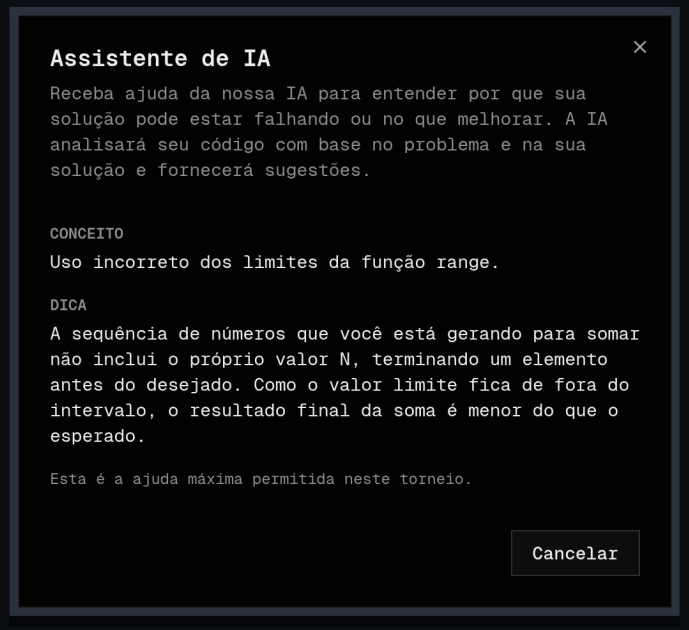
Quem monta
Professor monta a aula e comenta cada resposta

IA
Dica livre · código, blocos ou Parsons

O professor decide quanto a IA ajuda



Configuração do torneio: Desligada → Conceito → Dica → Precisa

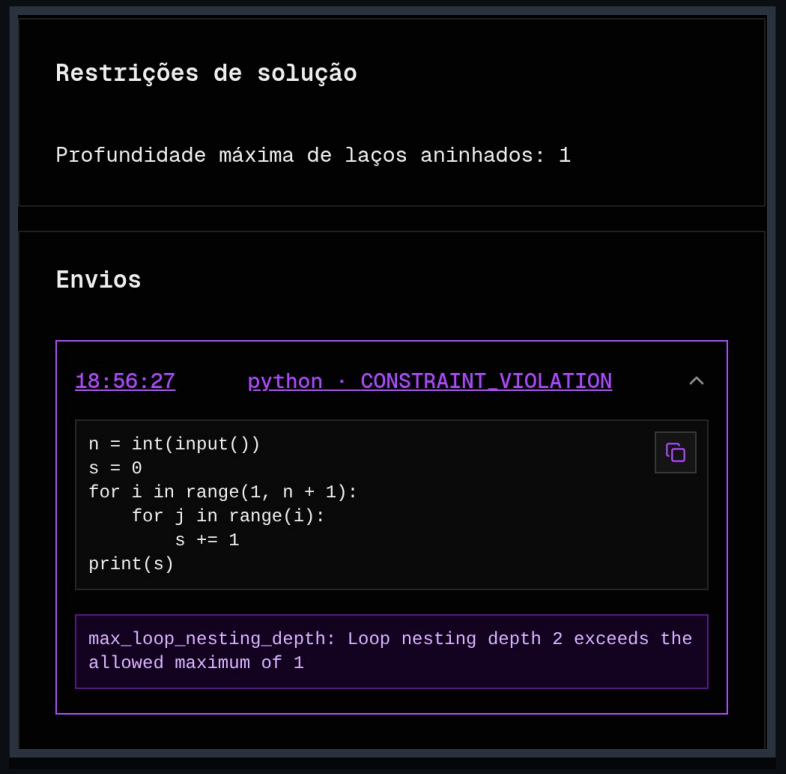


O aluno libera um nível por vez.
O servidor recusa qualquer pedido acima do limite do torneio, e uma mudança do professor vale já no próximo pedido.

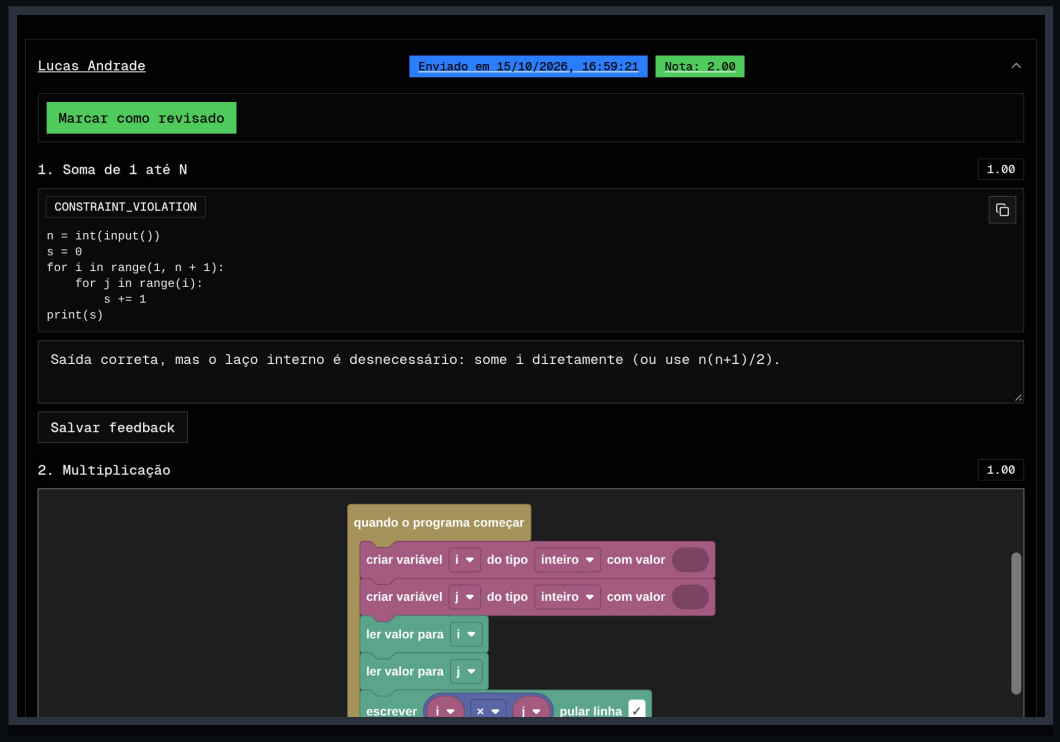
	IA DO WHYRUNNER	CHATBOT EXTERNO
Vê	código do aluno + teste que falhou	o que o aluno colar
Devolve	dica, sem a correção	solução pronta
Controle	professor, por torneio	nenhum
Limite	10 pedidos / 10 min	ilimitado
Formato	fala em blocos e linhas no modo visual	sempre código

O aluno ainda pode abrir outra IA. Por isso o professor também tem restrições de solução e atividades em sala.

O professor vê como o aluno resolveu, não só se acertou



Saída correta, mas viola "no máximo 1 laço"



Revisão do professor: nota pelos testes + comentário; resposta em blocos

Restrições
limite de laços aninhados, construções proibidas ou "deve usar Dijkstra" (checado por IA)

Formatos
código, blocos (gera Portugol) ou Parsons (ordenar linhas)

Revisão
após o prazo, nota automática e comentário do professor

O sistema está pronto; falta provar que ensina

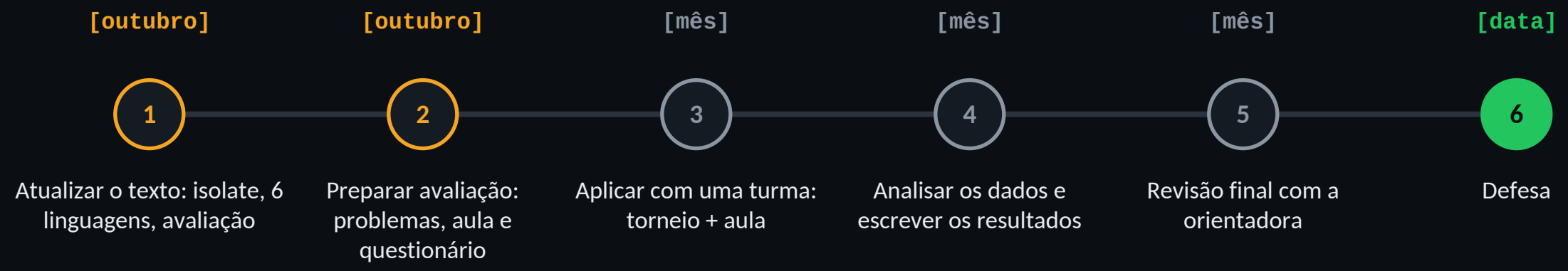
PONTOS FORTES

- \+ Sem broker de mensagens: só PostgreSQL
- \+ Isolamento no nível da IOI, sem Docker privilegiado
- \+ Tipagem de ponta a ponta (Drizzle + Zod)
- \+ IA sob demanda, com entrada delimitada contra prompt injection

LIMITAÇÕES

- ! Ainda sem avaliação com alunos
- ! Não impede consulta a IAs externas
- ! Análise de laços por texto, não por AST: falso positivo em strings
- ! Depende da API de IA (custo e limites) e de Linux com cgroups

Próximas etapas até a defesa



Etapa crítica: aplicar com a turma (3). Sem ela não há resposta para a pergunta de pesquisa; as etapas 4 e 5 dependem dela.

Referências

BROOKE, J. SUS: a quick and dirty usability scale. Usability Evaluation in Industry, 1996.

CAMPOS, C. P.; FERREIRA, C. E. BOCA: um sistema de apoio a competições de programação. WEI, 2004.

CARNEIRO, M. F. Sistema web para auxílio no treinamento para maratonas de programação. UTFPR, 2012.

DRIZZLE TEAM. Drizzle ORM: TypeScript ORM for SQL. 2024.

GEMINI TEAM, GOOGLE. Gemini: a family of highly capable multimodal models. 2023.

LIMA, G. M. Gamma Online Judge: plataforma para as questões das maratonas UnB de programação. 2022.

MAREŠ, M.; BLACKHAM, B. A new contest sandbox. Olympiads in Informatics, v. 6, 2012.

MATSAKIS, N. D.; KLOCK, F. S. The Rust language. ACM SIGAda Ada Letters, 2014.

MERKEL, D. Docker: lightweight Linux containers for consistent development and deployment. Linux Journal, 2014.

RUN CODES. Run Codes: plataforma de programação online. 2024.

SILVA, J. T. S. et al. Ataux: uma ferramenta para auxiliar o acompanhamento de submissões de questões de programação. 2021.

STONEBRAKER, M.; ROWE, L. A. The design of POSTGRES. ACM SIGMOD, 1986.

VERCEL INC. Next.js: the React framework for the web. 2024.